

深度學習hw2

(一) 演算法實作

本實驗分別實作了以下兩種強化學習演算法，並於相同環境中進行比較：

Q-learning (離策略方法, Off-policy)

Q-learning 透過維護並更新狀態-動作價值函數 $Q(s, a)$ 來學習最優策略。其更新公式如下：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

每次更新時，Q-learning 採用下一狀態所有可能動作中的最大 Q 值，即 $\max Q$ ，不論實際執行的是哪個動作。此機制使 Q-learning 能夠直接逼近最優策略的價值函數，不受行為策略中探索動作的干擾。

SARSA (同策略方法, On-policy)

SARSA 同樣維護並更新狀態-動作價值函數 $Q(s, a)$ ，其更新公式如下：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

與 Q-learning 不同，SARSA 使用「實際採取的下一個動作 a' 」來更新 Q 值，該動作同樣由 ϵ -greedy 策略選出。因此，SARSA 的 Q 值會將探索策略所帶來的風險納入評估，學到的策略能夠反映在當前行為策略下的實際表現。

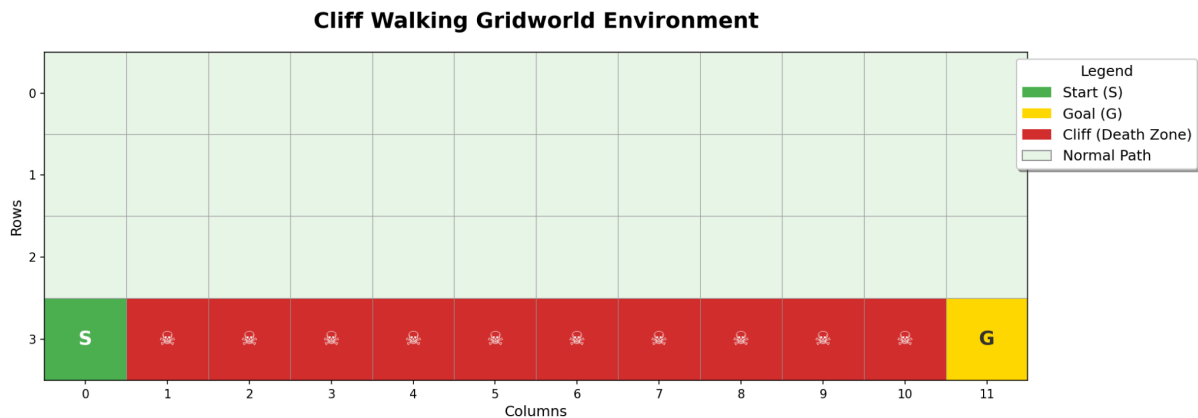
兩種演算法在完全相同的環境與參數設定下進行訓練，以確保比較的公平性。實驗設定如下：

- 環境：4 × 12 格子世界 (Cliff Walking)
- 學習率 (α)：0.1
- 折扣因子 (γ)：0.9
- 探索率 (ϵ)：0.1 (固定值)
- 訓練回合數：500 回合
- 獨立實驗次數：30 次 (用於統計分析)

兩種演算法皆採用 ϵ -greedy 策略進行動作選擇：以 $1 - \epsilon$ 的機率選取當前 Q 值最大的動作 (貪婪動作)，以 ϵ 的機率隨機選取其他動作 (探索動作)。兩者使用相同的 ϵ 值，確保探索程度一致。

獎勵機制設定如下：一般移動每步給予 -1 的懲罰；掉入懸崖給予 -100 的重大懲罰並回到起點；到達終點給予 -1 並結束回合。

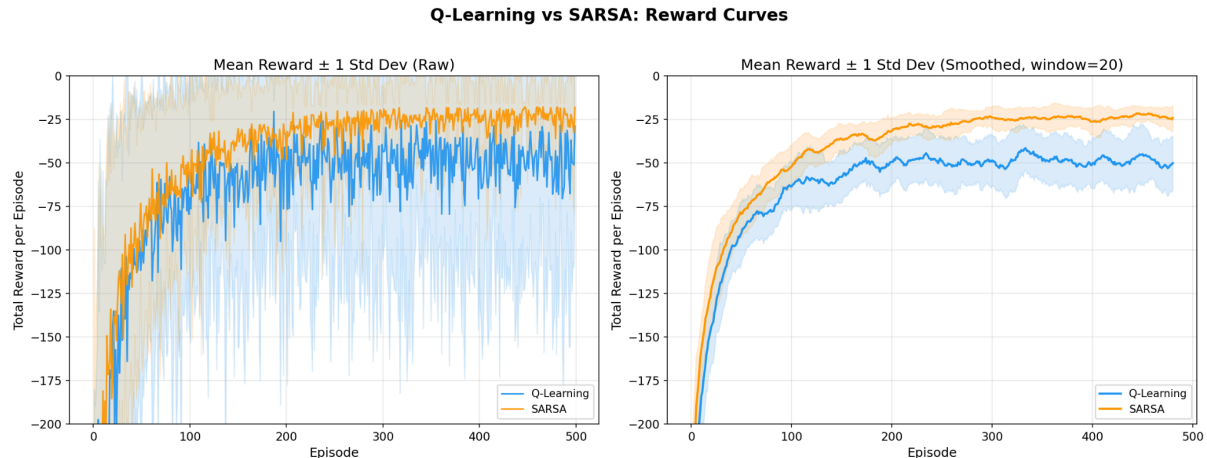
環境佈局如下圖所示，起點 S 位於左下角 (3, 0)，終點 G 位於右下角 (3, 11)，中間底部為懸崖區域 C：



(二) 結果分析

1. 學習表現

下圖展示了兩種演算法在 30 次獨立實驗中的平均累積獎勵曲線：



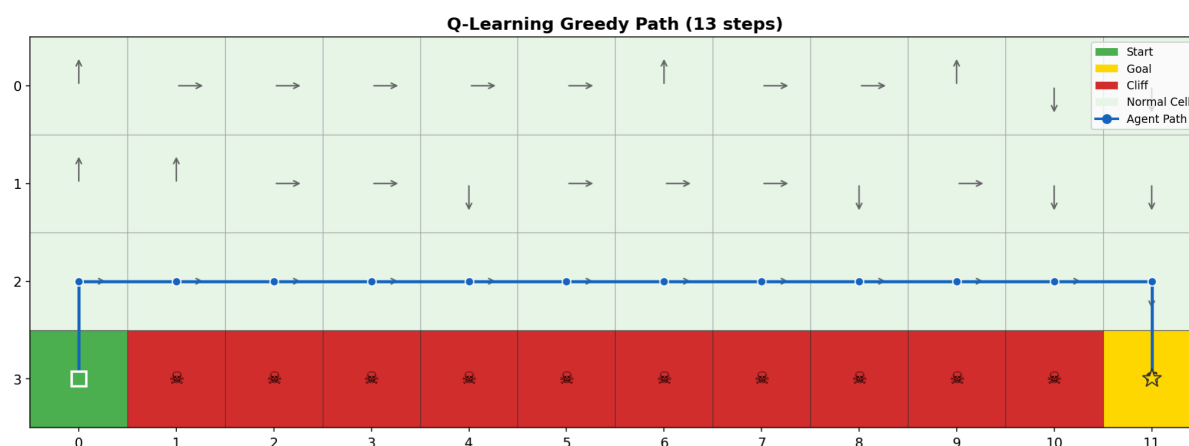
由圖可觀察到以下幾點：

SARSA 在約第 229 回合後達到穩定收斂，最後 50 回合的平均獎勵為 -23.49 ± 3.66 ，收斂後曲線平穩，波動幅度小。

Q-learning 在整個訓練過程中獎勵曲線波動明顯，最後 50 回合平均獎勵僅為 -48.34 ± 9.47 ，即便到第 500 回合仍未達到穩定收斂。

2. 策略行為

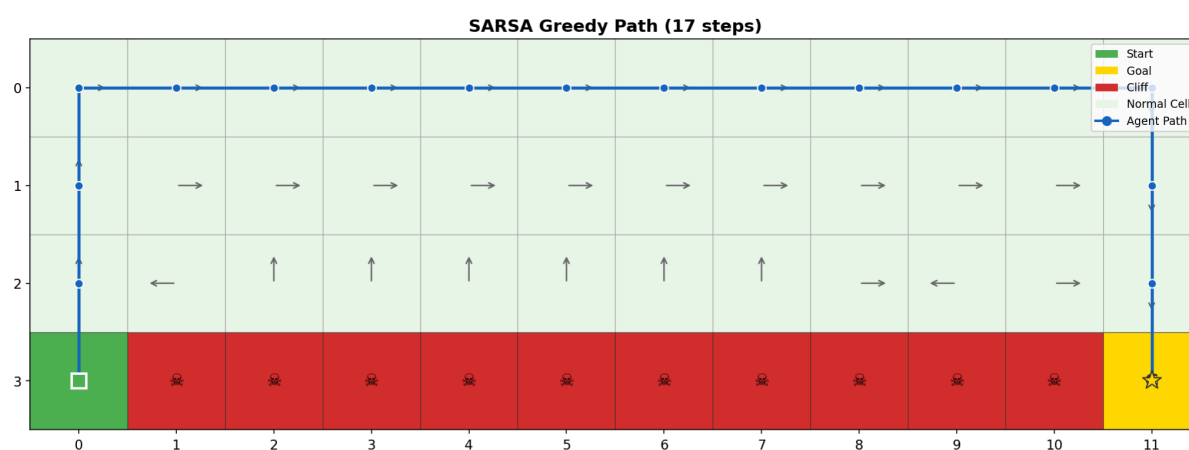
Q-learning 學習到的策略路徑(13 步):



路徑: $(3,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow \dots \rightarrow (2,11) \rightarrow (3,11)$

Q-learning 學到的是沿懸崖邊緣(第 2 列)直行的最短路徑, 共 13 步, 理論累積獎勵為 -13, 屬於理論上的最優路徑。然而在包含 ϵ -greedy 探索的實際執行中, 每一步均有 10% 的機率隨機移動, 極易掉入懸崖並承受 -100 的重大懲罰, 導致實際表現大幅劣化。這體現了 Q-learning 傾向冒險的特性。

SARSA 學習到的策略路徑(17 步):



路徑: $(3,0) \rightarrow (2,0) \rightarrow (1,0) \rightarrow (0,0) \rightarrow (0,1) \rightarrow \dots \rightarrow (0,11) \rightarrow (1,11) \rightarrow (2,11) \rightarrow (3,11)$

SARSA 選擇了一條繞行最頂部(第 0 列)的安全路徑, 共 17 步。雖然路徑較長, 但由於完全遠離懸崖, 即使在探索過程中發生隨機動作, 也不會掉入懸崖, 因此在 ϵ -greedy 策略下的期望總獎勵反而更高。這體現了 SARSA 傾向保守、安全的策略特性。

3. 穩定性分析

下表彙整了兩種演算法的量化穩定性指標：

指標	Q-Learning	SARSA
最後 50 回合平均獎勵	-48.34 ± 9.47	-23.49 ± 3.66
回合內標準差(平均)	67.37	20.80
變異係數(CV)	1.4059	0.8408
收斂回合(獎勵 > -30)	未穩定收斂	約第 229 回合
Greedy 路徑步數	13 步	17 步

從穩定性分析可得以下結論：SARSA 的變異係數(0.84)比 Q-learning(1.41)低約 40%，回合內獎勵波動亦低約 69%。Q-learning 即使在訓練後期，仍會因探索動作掉入懸崖而產生頻繁的大幅獎勵下跌，反觀 SARSA 的學習曲線在收斂後相對平穩。

探索(Exploration)對兩者的影響有本質上的差異。對 Q-learning 而言，探索動作雖不影響 Q 值的更新方向(因為更新始終使用 max Q)，但會在實際執行時帶來真實的懲罰，造成訓練期間的大幅波動。對 SARSA 而言，探索動作會直接反映在 Q 值的更新中，使靠近懸崖位置的 Q 值因多次掉入懲罰而降低，促使 agent 主動學習迴避高風險路線。

4. 使用情境分析

根據實驗中觀察到的「風險偏好」與「收斂路徑」差異，在實際應用中，選擇哪種演算法通常取決於以下幾個核心考量：

1. 什麼時候選擇 Q-learning？(追求極致效能)

Q-learning 作為一種 Off-policy 演算法，其目標是學習「理論上的最優策略」，而不考慮訓練過程中的探索風險。

- 離線學習(Offline Learning)：當你需要從歷史數據(別人錄製好的操作紀錄)中學習，而不是讓 Agent 親自去環境中闖蕩時，Q-learning 是首選。
- 容錯成本低、模擬環境：在純模擬的環境(如電子遊戲)中，Agent 掉入懸崖一萬次也沒關係，目標是訓練出那條「最短、最快」的 13 步路徑，此時 Q-learning 更有優勢。
- 部署時關閉探索：如果你的計畫是「訓練時容許失敗，部署時將 ϵ 設為 0(不進行隨機探索)」，那麼 Q-learning 最終能提供效率最高的決策。

2. 什麼時候選擇 SARSA？(追求安全性與穩定性)

SARSA 作為一種 On-policy 演算法，它在更新時會考慮到「下一個動作可能帶來的風險」，因此學習結果更為保守。

- 在線學習 (Online Learning) 與高風險環境：如果 Agent 是在現實世界中運行 (例如：一台價值百萬的昂貴機器人、無人機)，跌落或碰撞會造成實體損壞。SARSA 會學習到「遠離懸崖」的路徑，最大程度降低訓練過程中的損耗。
- 必須持續探索的場景：若應用環境中存在不確定性，導致 Agent 必須始終保持 $\epsilon > 0$ 的探索率，SARSA 學到的「安全路徑」產生的平均獎勵 (-23.49) 遠高於 Q-learning (-48.34)。
- 人類參與的系統：在推薦系統或醫療決策中，系統的「容錯率」極低，SARSA 的保守特性可以避免系統做出過於激進、可能導致災難性後果的決策。

特性	Q-learning (Off-policy)	SARSA (On-policy)
核心目標	理論最優解 (Shortest Path)	期望最優解 (Safest Path)
風險態度	冒險派、忽略探索帶來的懲罰	保守派、考慮探索動作的風險
最佳場景	模擬器、離線數據學習、追求極致效率	現實世界、昂貴硬體、追求訓練穩定
對 ϵ 的敏感度	即使 ϵ 很大也能學到最優路徑，但訓練過程波動劇烈	會隨著 ϵ 的大小調整策略，避免掉入陷阱